# Protecting the ALU Logo on the Ethereum Blockchain

Asset registration (ERC-721), tokenisation (ERC-20) and an exploration of a cryptocurrency exchange

---

Course: Cryptocurrency and Digital Assets

Student: Hauwa Muhammad Bello

African Leadership University

November 2026

## Abstract

This project protects the official African Leadership University (ALU) logo by registering it on an Ethereum blockchain. A Solidity contract, `ALUAssetRegistry`, records the logo as an ERC-721 NFT together with its SHA-256 fingerprint, the registering wallet and a timestamp, so anyone can prove whether a logo file is the authentic version or a tampered copy, while a second contract, `ALULogoToken`, splits ownership of the logo into 1,000,000 ERC-20 shares (ALUT) that ALU can distribute to its community. Eight automated tests confirm that both contracts behave as specified, and the report opens with an exploration of the Coinbase exchange that explains how digital assets are bought, sold and secured.

## 1. Introduction

ALU's logo signals that a certificate, admission letter, scholarship offer or event genuinely comes from the university. Anyone can copy a logo in seconds to make convincing fake documents or phishing pages, and modified versions can circulate until nobody is sure which file is official. ALU needs a way to prove, to anyone and at any time, which logo file is the real one and that ALU registered it first.

A **normal database** can store "this is our logo", but the record lives on servers that ALU controls. An administrator, an attacker or a software bug could change or delete it, and outsiders cannot check its history; they simply have to trust ALU's server. **Copyright and trademark registration** give legal rights, but they are territorial, slow and costly to enforce, and a certificate describes the mark in general terms rather than fingerprinting one exact digital file. Checking a file against a trademark register is a manual process.

A **blockchain** fills these gaps. The registration is copied across thousands of independent computers, cannot be edited once confirmed, carries a timestamp agreed by the whole network, and can be checked by anyone, anywhere, free of charge, without asking ALU. Because the record stores the file's SHA-256 hash, verification is exact: a file either matches bit for bit or it does not. The blockchain record does not replace legal protection. It is strong evidence that supports it, and it gives ALU something a database or trademark certificate cannot: a public, tamper-proof and instantly verifiable proof of the official logo and when it was registered. Tokenisation then adds a transparent way to share ownership of the logo with the ALU community.

# 2. Question 1: Exploring a Cryptocurrency Exchange (Coinbase)

> **Note:** This section answers Question 1 (697 words, excluding headings). The fee figures use Coinbase's published rates and the ETH price shown on Coinbase on 1 November 2026. Coinbase kept the Settings pages locked until proof of address was provided, so Steps 3, 5 and 6 are described without screenshots.

## Step 1: Creating an account

Signing up needs an email, a password and a phone number confirmed by SMS. To trade, Coinbase then asks for my legal name, date of birth and address, a photo of a government ID (passport, national ID or driver's licence) and a live selfie matched against the ID. My ID was verified in under ten minutes. It then asked for proof of address: a dated document with my full legal name and residential address, such as a bank statement. My ID card does not show my address, so I stopped there.

This is Know Your Customer (KYC). Crypto crosses borders in minutes and transfers cannot be reversed, so anonymous accounts would be ideal for money laundering, terrorist financing and fraud with stolen identities. In the United States, KYC is required by the **Bank Secrecy Act**: FinCEN treats exchanges as money services businesses that must verify customers, run anti-money-laundering (AML) programmes and report suspicious activity. In Rwanda, **Law No. 023/2026 regulating virtual asset business** requires licensed providers to have AML controls, supervised by the Capital Market Authority.

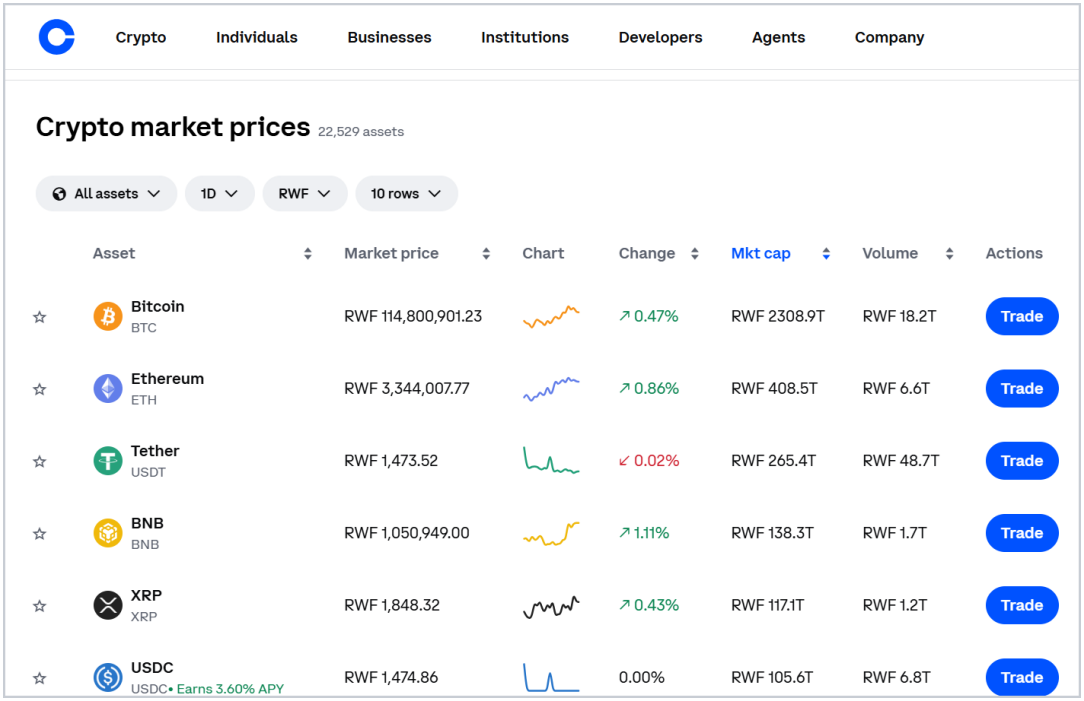



**Screenshot 1.** *Coinbase identity verification: checking my ID (left) and ID verified (right).*

## Step 2: Exploring the platform

Without full verification I could browse only the public pages. The Explore page lists over 22,000 assets with price, daily change, market cap, volume and a Trade button, in Rwandan francs. Verified users also get a dashboard of balances, a wallet with Send and Receive, and Buy, Sell and Convert screens. Three listed assets:

- **Bitcoin (BTC)**, the first cryptocurrency (2009), is used as peer-to-peer digital money and a store of value. Its supply is permanently capped at 21 million coins.
- **Ethereum (ETH)** is a programmable blockchain whose coin pays the gas fees for transactions and smart contracts. Unlike the others, anyone can deploy programs on it, as I did in Question 2.
- **USD Coin (USDC)** is a stablecoin issued by Circle, backed by cash and short-term US government debt, and used for payments and holding value between trades. Unlike BTC and ETH, its price is designed to stay at US$1.




**Screenshot 2.** *Coinbase Explore page (prices in Rwandan francs) listing Bitcoin, Ethereum and USDC.*

## Step 3: Adding a payment method

Under Settings → Payment methods, a bank account is linked by signing in through the bank or entering account and routing numbers, in my verified name. A debit card needs the card number, expiry date, CVV and billing address, often confirmed with a one-time code. The **debit card is faster** (purchases complete instantly); the **bank transfer is cheaper** (about 1.49% versus 3.99% for cards) but can take several business days to settle.

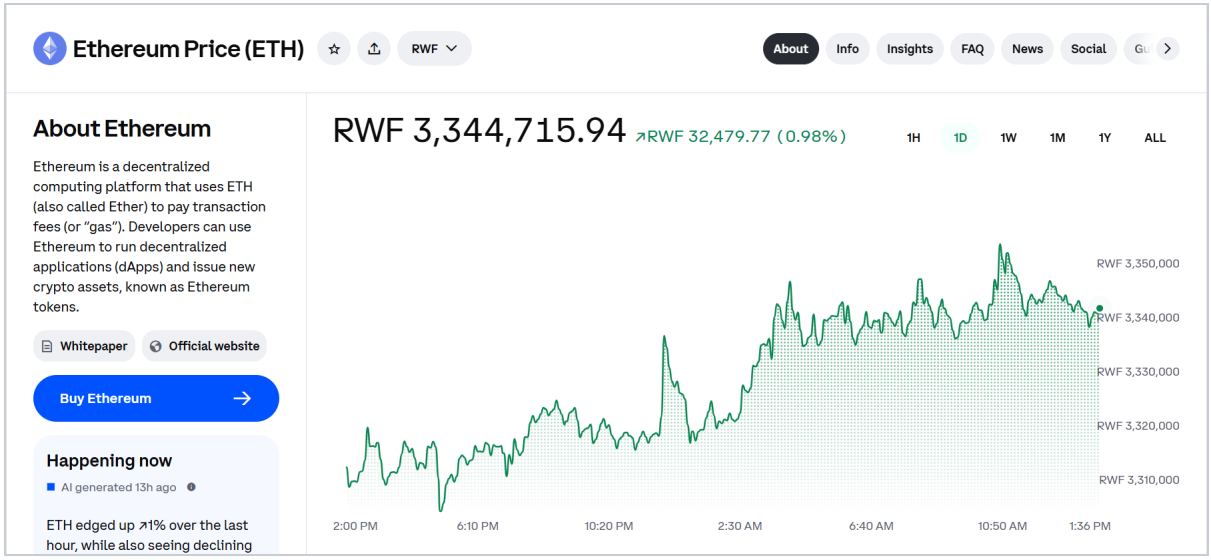
## Step 4: Simulating a $100 purchase

A **market order** buys immediately at the best available price: it always fills, but the price is not guaranteed. A **limit order** only fills at the price I set or better, so I control the price but the order may never execute.

I chose a **market order** for ETH: $100 is small, ETH trades so heavily that slippage is negligible, and I wanted it completed straight away.

On 1 November 2026 Coinbase showed ETH at RWF 3,344,716, about $2,268. Coinbase charges the greater of a flat fee or a percentage. By debit card that is 3.99% ($3.99), which leaves $96.01, and a spread of roughly 0.5% raises the price to about $2,279. I would receive 96.01 ÷ 2,279 ≈ **0.0421 ETH**, worth $95.53 at market price, so fees cost about $4.47.

| | Debit card | Bank transfer |
|---|---|---|
| Amount spent | $100.00 | $100.00 |
| Coinbase fee | $3.99 (3.99%) | $2.99 (flat fee) |
| Amount converted | $96.01 | $97.01 |
| Price paid incl. ~0.5% spread | ≈ $2,279 / ETH | ≈ $2,279 / ETH |
| **ETH received** | **≈ 0.0421 ETH** | **≈ 0.0426 ETH** |




**Screenshot 3.** *Coinbase Ethereum price page on 1 November 2026: RWF 3,344,715.94, about $2,268 with USDC at RWF 1,474.86.*

## Step 5: Reviewing the transaction record

Each entry records the type (buy, sell, send, receive, convert), asset, amount, price, fee, payment method, date and status; sends and receives add the address, network fee and transaction hash.

A **transaction hash** is a unique 64-character ID produced by hashing the transaction's data. Entering it in a block explorer such as Etherscan proves the transaction happened and shows its confirmations and fee, which helps track transfers and settle disputes. Purchases inside Coinbase are only internal ledger entries, so they get a hash only when coins move on-chain.

## Step 6: Security features

- **Two-step verification (2FA)** requires a second proof after the password: an SMS code, authenticator app, push prompt, passkey or security key. A stolen password alone is then not enough. Coinbase ranks SMS weakest because of SIM-swap attacks.
- **Address book allowlisting** lets withdrawals go only to addresses I have saved in advance. Changing the setting requires 2FA, and after a short setup window new addresses face a 48-hour hold. Crypto transfers are irreversible, so this stops an intruder quickly draining funds to their wallet.

# 3. Blockchain Setup (Part A)

Before writing contracts I needed a blockchain to deploy them on. I used **Hardhat**, which runs a complete Ethereum network on my own computer. The table below explains what each part of the setup does.

| Component | What it is doing on my machine |
|---|---|
| **Node.js 22 LTS** | The JavaScript runtime that runs Hardhat, the deployment scripts and the tests. |
| **npm** | The package manager. It downloads the libraries listed in `package.json` and pins their exact versions in `package-lock.json`. |
| **Hardhat 2.29.1** | The Ethereum development environment. It compiles Solidity, runs tests, runs scripts and provides **Hardhat Network**, the local blockchain. |
| **Hardhat Toolbox 6.1.2** | A bundle of plugins: ethers.js v6 (to talk to the blockchain from JavaScript), Mocha and Chai matchers (testing), network helpers (time control, snapshots), a gas reporter and coverage. |
| **OpenZeppelin Contracts 5.6.1** | Audited, standard implementations of ERC-721, ERC-20 and Ownable, so I do not write security-critical code from scratch. |
| **hardhat.config.js** | Sets the Solidity compiler (0.8.28, optimizer on, EVM target `cancun`) and the networks (`localhost` at 127.0.0.1:8545, chain ID 31337). |
| **contracts/, scripts/, test/** | The Solidity source code, the deployment scripts and the automated tests. Compiling creates `artifacts/` (the ABI and bytecode of each contract) and `cache/`. |

## 3.1 Installation steps

```
node -v                                        # check that Node.js 22 is installed
mkdir alu-logo-registry && cd alu-logo-registry
npm init -y                                    # create package.json
npm install --save-dev hardhat@2.29.1 @nomicfoundation/hardhat-toolbox@6.1.2
npx hardhat init                               # choose "Create a JavaScript project"
npm install @openzeppelin/contracts@5.6.1      # ERC-721, ERC-20 and Ownable
npx hardhat compile                            # compile the contracts
npx hardhat node                               # start the local blockchain
```

`npx hardhat node` starts a JSON-RPC server at `http://127.0.0.1:8545`. Wallets and scripts send transactions to this address exactly as they would to a real Ethereum node. It creates **20 test accounts**, each funded with 10,000 test ETH, and mines a new block instantly for every transaction. Figure 1 shows the node running with its generated wallet addresses.

```
evolver@DESKTOP-0B2I2HO:~/alu-logo-registry$ npx hardhat node
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/

Accounts
========

WARNING: These accounts, and their private keys, are publicly known.
Any funds sent to them on Mainnet or any other live network WILL BE LOST.

Account #0: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266 (10000 ETH)
Private Key: 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80

Account #1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000 ETH)
Private Key: 0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d

Account #2: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC (10000 ETH)
Private Key: 0x5de4111afa1a4b94908f83103eb1f1706367c2e68ca870fc3fb9a804cdab365a

Account #3: 0x90F79bf6EB2c4f870365E785982E1f101E93b906 (10000 ETH)
Private Key: 0x7c852118294e51e653712a81e05800f419141751be58f605c371e15141b007a6

Account #4: 0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65 (10000 ETH)
Private Key: 0x47e179ec197488593b187f80a00eb0da91f1b9d0b13f8733639f19c30a34926a

Account #5: 0x9965507D1a55bcC2695C58ba16FB37d819B0A4dc (10000 ETH)
Private Key: 0x8b3a350cf5c34c9194ca85829a2df0ec3153be0318b5e2d3348e872092edffba

Account #6: 0x976EA74026E726554dB657fA54763abd0C3a0aa9 (10000 ETH)
Private Key: 0x92db14e403b83dfe3df233f83dfa3a0d7096f21ca9b0d6d6b8d88b2b4ec1564e

Account #7: 0x14dC79964da2C08b23698B3D3cc7Ca32193d9955 (10000 ETH)
Private Key: 0x4bbbf85ce3377467afe5d46f804f221813b2bb87f24d81f60f1fcdbf7cbf4356

Account #8: 0x23618e81E3f5cdF7f54C3d65f7FBc0aBf5B21E8f (10000 ETH)
Private Key: 0xdbda1821b80551c9d65939329250298aa3472ba22feea921c0cf5d620ea67b97

Account #9: 0xa0Ee7A142d267C1f36714E4a8F75612F20a79720 (10000 ETH)
Private Key: 0x2a871d0798f97d79848a013d4936a73bf4cc922c825d33c1cf7073dff6d409c6
```

**Figure 1.** *The local Hardhat node running on my machine, showing the generated test wallet addresses (accounts #0 to #9 of 20).*

## 3.2 What is a local blockchain and why do developers use one?

A local blockchain is a private copy of the Ethereum system running on a single computer. It follows the same rules as the real network: the same virtual machine (EVM), the same transaction format and the same smart-contract behaviour. But only one node exists, and it belongs to the developer. Developers use it during development because:

- **It is free.** Every transaction on Ethereum mainnet costs real ETH in gas. Locally, the test accounts hold 10,000 fake ETH each.
- **It is instant.** Blocks are mined immediately instead of every 12 seconds, so tests and deployments run in under a second.
- **Mistakes are harmless.** Contracts on mainnet are permanent, so a bug there could lock or lose funds forever. Locally I can fix the code, restart the node and redeploy as often as I like.
- **It is easier to debug.** Hardhat shows clear revert reasons and stack traces, and tests can move time forward or take snapshots.
- **It is private and works offline.** Nothing is published until the code is ready.

## 3.3 What is a wallet address?

A wallet address is the public identifier of an account on the blockchain, written as `0x` followed by 40 hexadecimal characters, for example `0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266` (account #0 above). It is derived from a **private key**: the private key produces a public key using elliptic-curve

cryptography, and the address is the last 20 bytes of the Keccak-256 hash of that public key. Because this only works one way, the address can be shared freely while the private key stays secret.

An address represents **an identity that can own things and act**. It can hold ETH, tokens and NFTs, and any transaction signed with its private key is accepted as coming from that address. Smart contracts have addresses too; my registry was deployed at `0x5FbDB2315678afecb367f032d93F642f64180aa3`. In this project the deployer's address is recorded as the registrant of the logo, owns NFT #1 and receives all 1,000,000 ALUT. The Hardhat private keys are publicly known, so they must never be used on a real network.

# 4. Smart Contract Explanation (Part B)

## 4.1 Generating the ALU logo hash

I downloaded the official full-colour ALU logo from the ALU website (`alueducation.com/wp-content/uploads/2023/05/ALU-logo-with-name-min.png`, a 1258 × 228 PNG of 11,277 bytes) and saved it unchanged as `alu-logo.png`. I generated its hash programmatically with Node.js using `scripts/hash-logo.js`. The script reads the file's raw bytes, passes them to `crypto.createHash("sha256")` and prints the 64-character result, both on its own and with the `0x` prefix that a Solidity `bytes32` parameter needs. I cross-checked the result with the operating system's `sha256sum` tool (Figure 2).

> **ALU logo SHA-256:**
> `f45d877814a52c87c1c341df66ac2ccb4e715074bd0fe90ca435807eb310f4da`

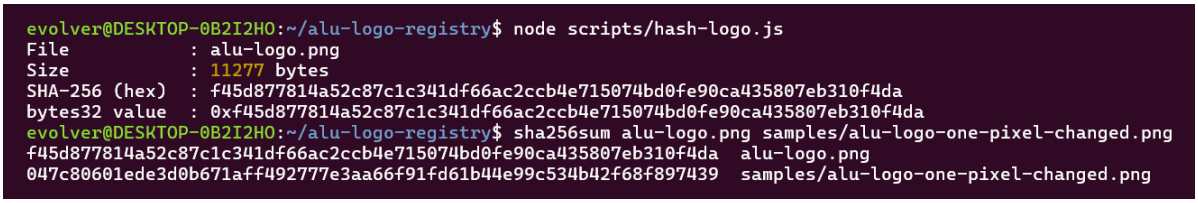



**Figure 2.** *Generating the SHA-256 hash of the real ALU logo with Node.js on my machine, cross-checked with sha256sum. The second file is a copy with one pixel changed.*

This value is passed as the `contentHash` argument when the logo is registered. The deployment script also recomputes the hash from `alu-logo.png` and refuses to deploy if the file does not match, so the hash on the blockchain always belongs to the real file.

## 4.2 ALUAssetRegistry: data structure, mappings and event

`ALUAssetRegistry` extends OpenZeppelin's `ERC721` contract, so it is a full NFT collection (name "ALU Asset Registry", symbol "ALUAR") with standard ownership and transfer functions. On top of that I added the following.

```
struct AssetMetadata {
    string  assetName;     // e.g. "ALU Official Logo"
    string  fileType;      // e.g. "image/png"
    bytes32 contentHash;   // SHA-256 hash of the file
    address registeredBy;  // wallet that registered the asset
    uint256 registeredAt;  // Unix timestamp of the registration block
}
mapping(uint256 => AssetMetadata) public assets;            // token ID -> details
mapping(bytes32 => bool)          public registeredHashes;  // hash -> already used?
```

- **AssetMetadata** groups the five facts recorded for every asset. `registeredBy` is taken from `msg.sender` and `registeredAt` from `block.timestamp`, so neither can be faked by the caller.
- **assets** (mapping 1) links each token ID to its metadata. It is `public`, so anyone can read it.
- **registeredHashes** (mapping 2) remembers which file hashes have been used, so the same file can never be registered twice.
- **_tokenIdCounter** is a private counter that gives each new NFT the next ID, starting at 1.

- **AssetRegistered** is an event emitted on every registration. It records the token ID, registrant, hash, name, file type and timestamp in the transaction log so apps can listen for new registrations.

### 4.3 Function 1: registerAsset()

`registerAsset(string assetName, string fileType, bytes32 contentHash) returns (uint256)` is the function that puts an asset on the blockchain. In plain English, it:

1. Checks that the hash is not empty, and that the hash has **not been registered before**. If it has, the whole transaction is cancelled with the message "ALUAssetRegistry: this file has already been registered".
2. Increments the counter to get the new token ID.
3. Saves the asset name, file type, hash, the caller's address and the current time in the `assets` mapping.
4. Marks the hash as used in `registeredHashes`.
5. Mints the new NFT to the caller with `_safeMint()`. The safe version checks that a contract receiver can actually handle NFTs, so the token cannot get stuck.
6. Emits the `AssetRegistered` event and returns the new token ID.

> **Design decision:** the assignment lists minting before saving the metadata. I save the metadata and mark the hash as used **before** calling `_safeMint()`, following the checks-effects-interactions pattern. `_safeMint()` calls back into the receiver if it is a contract, and if the hash were not yet marked as used, a malicious contract could re-enter `registerAsset()` during that callback and register the same file twice. Every required step still happens in the same transaction.

### 4.4 Function 2: verifyLogoIntegrity()

`verifyLogoIntegrity(uint256 tokenId, bytes32 contentHash) returns (bool, string)` lets anyone check a file. The caller hashes the file they have, then passes the token ID and that hash. The function looks up the hash stored for the token and compares the two. If they are equal it returns `true` and "Logo is authentic."; if not, it returns `false` and "Warning: logo does not match.". It is a `view` function, meaning it only reads data, so calling it is free: no transaction is sent and no gas is paid. I also made it revert with "token does not exist" for IDs that were never registered. Otherwise an unregistered token, whose stored hash is all zeros, could wrongly "match" a zero hash.

### 4.5 Function 3: getAsset()

`getAsset(uint256 tokenId) returns (AssetMetadata)` is a free `view` function that returns the whole registration record for a token: its name, file type, hash, registrant and timestamp. Anyone can use it at any time to see exactly what was registered, by whom and when. As a small extra, `totalAssets()` returns how many assets have been registered, which the future dApp can use to list them.

## 4.6 Deployment and verification scripts

`scripts/deploy.js` deploys `ALUAssetRegistry` and immediately calls `registerAsset("ALU Official Logo", "image/png", <logo hash>)`, so the logo is registered as part of deployment. It reads the new token ID from the `AssetRegistered` event, confirms integrity with `verifyLogoIntegrity()`, deploys `ALULogoToken` (Section 5) and saves the addresses to `deployments/localhost.json` for the next assignment. Figure 3 shows a deployment to the local node.

```
evolver@DESKTOP-0B2I2HO:~/alu-logo-registry$ npx hardhat run scripts/deploy.js --network localhost
Network  : localhost
Deployer : 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266

ALU logo SHA-256 : 0xf45d877814a52c87c1c341df66ac2ccb4e715074bd0fe90ca435807eb310f4da
ALUAssetRegistry deployed to: 0x5FbDB2315678afecb367f032d93F642f64180aa3
ALU logo registered as token ID: 1
  tx hash       : 0x12575cabdbf10226d97aa02de33cc11e532882bdb9373de8fbc8d0afaf9addd2
  asset name    : ALU Official Logo
  file type     : image/png
  registered by : 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
  registered at : 2026-11-01T10:43:24.000Z
  integrity     : true - Logo is authentic.

ALULogoToken deployed to: 0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0
  name / symbol : ALU Logo Token / ALUT
  total supply  : 1000000.0 ALUT
  owner balance : 1000000.0 ALUT
  owner share   : 100%

Addresses saved to deployments/localhost.json
```

**Figure 3.** *Running scripts/deploy.js against the local node on my machine. The ALU logo is registered as token ID 1 and the full ALUT supply is minted to the deployer.*

To demonstrate the real-world use case, `scripts/verify-logo.js` hashes any file and checks it against the deployed registry. The genuine logo is confirmed as authentic, while a copy with a single pixel changed is rejected (Figure 4).

```
evolver@DESKTOP-0B2I2HO:~/alu-logo-registry$ npx hardhat run scripts/verify-logo.js --network localhost
File checked   : alu-logo.png
File SHA-256   : 0xf45d877814a52c87c1c341df66ac2ccb4e715074bd0fe90ca435807eb310f4da
Token ID       : 1
Result         : true - Logo is authentic.
evolver@DESKTOP-0B2I2HO:~/alu-logo-registry$ LOGO_FILE=samples/alu-logo-one-pixel-changed.png npx hardhat run scripts/verify-logo.js --network localhost
File checked   : samples/alu-logo-one-pixel-changed.png
File SHA-256   : 0x047c80601ede3d0b671aff492777e3aa66f91fd61b44e99c534b42f68f897439
Token ID       : 1
Result         : false - Warning: logo does not match.
```

**Figure 4.** *Verifying files against the blockchain on my machine: the real logo is authentic, the one-pixel-changed copy is flagged.*

## 4.7 Key concepts

### What is a smart contract, and how is it different from a normal program on a server?

A smart contract is a program whose code and data are stored on a blockchain at their own address. When someone sends it a transaction, every node in the network runs the same code and must reach the same result, so the contract always does exactly what its code says. A normal program on a server differs in several ways:

| | Program on a server | Smart contract |
|---|---|---|
| Control | The owner can change the code or data, or switch it off, at any time. | The code cannot be changed after deployment, and no single party can stop it. |
| Where it runs | On one organisation's machines. | On thousands of independent nodes at once. |

| | Program on a server | Smart contract |
|---|---|---|
| Trust | Users must trust the operator. | Anyone can read the code and check every result. |
| Record | Logs are private and editable. | Every transaction is public and permanent. |
| Cost | Free for users; the operator pays for hosting. | Changing state costs gas; reading (view functions) is free. |

### What is an NFT, and why is ERC-721 the right standard for a logo?

An NFT (non-fungible token) is a token that is **unique and not interchangeable**: each one has its own ID, its own data and exactly one owner. ERC-721 is the Ethereum standard (EIP-721) that defines how NFTs behave, including `ownerOf(tokenId)`, safe transfers, approvals and `Transfer` events. It is the right choice for the logo because the logo is one specific asset. ERC-721 gives it a unique token ID (#1) tied to its own fingerprint and details, a single clearly identifiable owner at any moment, and standard transfer rules. Because every wallet, block explorer and marketplace understands ERC-721, the registration can also be displayed and checked by existing tools, and by the dApp in the next assignment. A fungible standard such as ERC-20 could not do this, because every unit is identical and none can carry its own hash.

### What is a SHA-256 hash, and why does changing one pixel produce a completely different hash?

SHA-256 is a cryptographic hash function from the SHA-2 family (standardised by NIST in FIPS 180-4). It turns any input, whether a word or a whole video, into a fixed 256-bit value, written as 64 hexadecimal characters. It is **deterministic** (the same file always gives the same hash), **one-way** (the file cannot be rebuilt from the hash) and **collision-resistant** (finding two different files with the same hash is practically impossible). This makes the hash a reliable fingerprint of a file.

The hash is calculated from the file's **bytes**, not from how the image looks. Changing one pixel changes at least one byte, and in a compressed PNG it usually changes many, because the compressed data shifts. SHA-256 then mixes the data through 64 rounds of bit rotations, XORs and additions, in which every input bit affects many output bits. This **avalanche effect** means that flipping even a single input bit changes about half of the output bits, so the new hash bears no resemblance to the old one. Figure 5 shows this with the real logo: one pixel at position (120, 60) was recoloured from red to grey, a change invisible at normal size, and the hash changed from `f45d87…f4da` to `047c80…7439`.

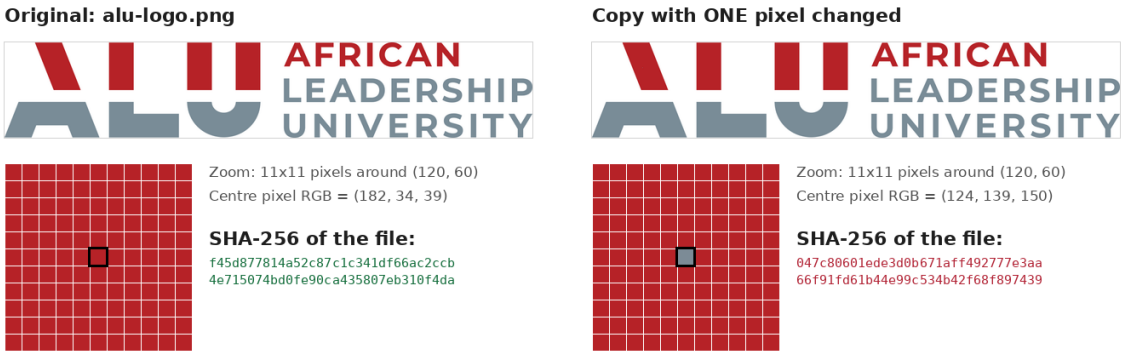



**Figure 5.** *The real ALU logo and a copy with exactly one pixel changed. The images look identical, but their SHA-256 hashes are completely different.*

A practical consequence is that even re-saving the logo in another format or compression level gives a different hash. ALU should therefore publish and distribute the exact registered file.

### Why can the data not be changed once it is stored on the blockchain?

- **Blocks are chained by hashes.** Each block contains the hash of the block before it. Altering the logo record in an old block would change that block's hash and break the link to every later block, so tampering is immediately obvious.
- **Thousands of nodes hold copies and must agree.** To rewrite history, an attacker would have to overpower the honest majority of the network. On Ethereum's proof-of-stake system that would mean controlling an enormous amount of staked ETH, which would be destroyed (slashed) if they were caught. This is economically unrealistic.
- **The contract allows no edits.** I deliberately wrote no update or delete function. Once a hash is stored and marked as used, not even the deployer can change it, and the contract's code itself cannot be modified after deployment.
- **An honest caveat:** my local Hardhat network has only one node, on my laptop, so I could simply restart it. The permanence described above comes from deploying the same contracts to a public network such as Ethereum mainnet.

> **Trust anchor:** anyone can call `registerAsset()`, so an impostor could register a different file and name it "ALU Official Logo". What makes the record authoritative is that token #1 was registered first, by ALU's own wallet (`registeredBy`), and the duplicate check stops anyone re-registering the real file. Restricting registration to authorised ALU wallets is listed as an improvement in Section 7.

# 5. Tokenisation (Part C)

## 5.1 What is tokenisation?

Tokenisation means representing ownership rights in an asset as digital tokens on a blockchain. Instead of one person owning the whole asset, its ownership is divided into many identical units that can be held by different people, transferred instantly and tracked publicly, with the rules enforced by code. Real-world examples include fractional shares of buildings, bonds and artworks. In this project the logo NFT remains one indivisible asset, and `ALULogoToken` divides its ownership into **1,000,000 equal ALUT shares**.

## 5.2 The ALULogoToken contract

- **Token setup.** `ALULogoToken` extends OpenZeppelin's `ERC20` and `Ownable`. Its name is "ALU Logo Token", its symbol "ALUT", and it uses the standard 18 decimals. `TOTAL_SUPPLY` is a constant of 1,000,000 tokens.
- **Constructor.** `constructor(address logoOwner)` mints the whole supply to the logo owner and makes them the contract owner. There is no other mint function, so the supply is fixed forever.
- **Function 4: distributeShares(recipient, amount).** Restricted with `onlyOwner`, so only the logo owner can call it. It requires `amount > 0`, then transfers that many ALUT from the owner's balance to the recipient and emits `SharesDistributed`. This is how ALU would hand shares to faculty, students or administration. Amounts use 18 decimals, so 100,000 ALUT is written `ethers.parseUnits("100000", 18)`.
- **Function 5: ownershipPercentage(account).** A free `view` function that returns `balance × 100 ÷ totalSupply` as a whole number: 500,000 ALUT returns 50. Integer division rounds down, so holdings below 1% return 0.
- **Deployment update.** `scripts/deploy.js` deploys `ALULogoToken` after the registry, passing the deployer's address as `logoOwner`, so the deployer receives the full 1,000,000 ALUT (Figure 3).

## 5.3 Who at ALU would hold ALUT tokens, and why?

The allocation below is an illustrative example of how ALU could distribute the 1,000,000 ALUT. The administration keeps a majority so that control of the brand stays with the university, while the community that gives the logo its value receives a meaningful share.

| Stakeholder | Example ALUT | Share | Why they would hold tokens |
|---|---|---|---|
| ALU administration / university treasury | 600,000 | 60% | Remains the majority owner and guardian of the brand; approves official uses of the logo. |
| Faculty | 100,000 | 10% | Their teaching and research build the reputation that the logo stands for. |
| Students (through the student guild) | 100,000 | 10% | Gives students a voice in how the brand is used, for example on merchandise and events. |
| Alumni network | 100,000 | 10% | Alumni carry the brand into industry; tokens keep them connected and invested in its reputation. |

| Stakeholder | Example ALUT | Share | Why they would hold tokens |
|---|---|---|---|
| Reserve for staff and future partners | 100,000 | 10% | Can reward staff or be offered to partners in co-branding agreements. |

The tokens become meaningful when ALU attaches rights to them, such as a vote on logo-usage requests weighted by holdings, or a pro-rata share of revenue from licensing the logo for merchandise. Because rights like these can make tokens resemble investments, a real launch would need legal advice. Rwanda's 2026 virtual asset law also covers token issuance, so the Capital Market Authority might need to approve it.

## 5.4 ERC-721 versus ERC-20

| | ERC-721 (ALUAssetRegistry) | ERC-20 (ALULogoToken) |
|---|---|---|
| Fungibility | Non-fungible: every token is unique. | Fungible: every token is identical and interchangeable. |
| How tokens are identified | By token ID: which one you own. | By balance: how many you own. |
| Divisibility | Indivisible: you own the whole token or none of it. | Divisible: 18 decimals, down to 0.000000000000000001 ALUT. |
| Key functions | `ownerOf`, `safeTransferFrom`, `approve` | `balanceOf`, `transfer`, `totalSupply`, `approve` |
| Typical uses | Certificates, titles, collectibles, unique assets. | Currencies, shares, reward points. |
| In this project | Token #1 is the ALU logo itself, with its hash and metadata. | 1,000,000 ALUT are the ownership shares of that logo. |

I used **ERC-721 for the registration** because the logo is a single, unique asset that needs its own ID and its own fingerprint. I used **ERC-20 for the ownership shares** because shares are identical: one ALUT is worth exactly the same as any other, and people need to hold different quantities and see their balance and percentage. Using ERC-721 for the shares would require a million separate NFTs, and ERC-20 alone could not attach a file hash to the logo. The two standards complement each other: the NFT says **what** the asset is, and the tokens say **who owns how much** of it.

## 5.5 What does holding 100,000 ALUT mean in practice?

A faculty member holding 100,000 of the 1,000,000 ALUT owns **10% of the logo's ownership shares**, and `ownershipPercentage()` returns 10 for their address (this exact case is checked in test 8). In practice:

- If ALUT holders vote on logo-usage decisions, they control 10% of the votes. That is a real voice, but a minority one, since the administration's 60% still decides.
- If ALU shares logo licensing income in proportion to holdings, they receive 10% of it. For example, $2,000 of $20,000 earned from merchandise in a year.
- They can transfer or sell some of their tokens with a standard ERC-20 transfer. Giving 20,000 ALUT to a colleague would lower their share to 8%.

- Their stake is public, provable and cannot be taken away without their own signature. On its own, though, the token does not transfer copyright. Its legal meaning comes from the policy or agreement ALU attaches to it.

## 6. Test Results

The test file `test/ALUAssetRegistry.test.js` contains eight automated tests written with Mocha, Chai and Hardhat's network helpers. Each test starts from a fresh deployment using `loadFixture` snapshots. The hashes are computed from the real files: the actual ALU logo, and the copy with one pixel changed for the negative test. Running `npx hardhat test` passes all eight (Figure 6).

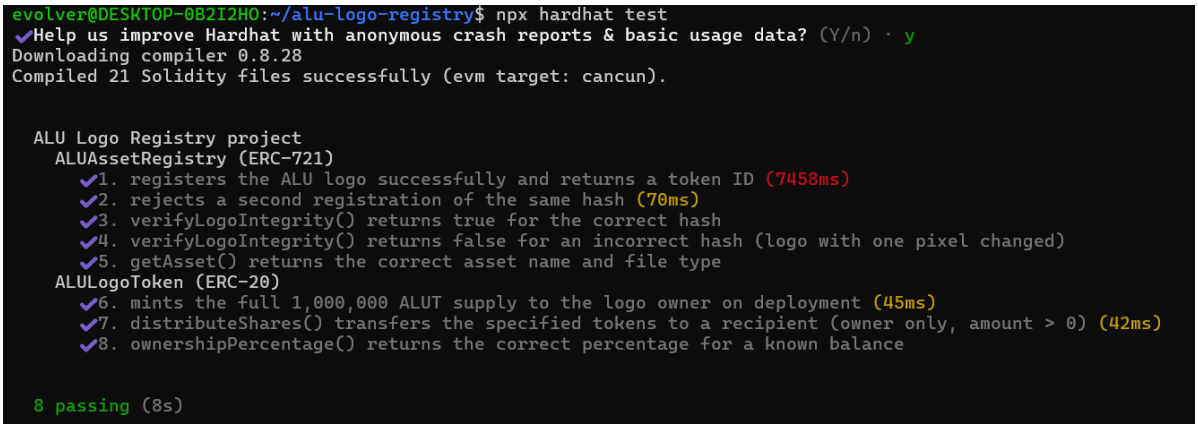



**Figure 6.** *All eight automated tests passing on my own machine (npx hardhat test in the Ubuntu terminal).*

| # | Test | What it proves |
|---|---|---|
| 1 | The ALU logo registers successfully and returns a token ID | `registerAsset()` returns 1, emits `AssetRegistered` with the right data and mints NFT #1 to the caller. |
| 2 | Registering the same hash a second time is rejected | The duplicate is reverted with a clear message, even when a different wallet tries it. |
| 3 | `verifyLogoIntegrity()` returns true for the correct hash | Returns `true` and "Logo is authentic." for the real logo's hash. |
| 4 | `verifyLogoIntegrity()` returns false for an incorrect hash | Returns `false` and "Warning: logo does not match." for the one-pixel-changed copy. |
| 5 | `getAsset()` returns the correct asset name and file type | Returns "ALU Official Logo" and "image/png", plus the correct hash, registrant and timestamp. |
| 6 | The full 1,000,000 ALUT supply is minted to the logo owner | Name, symbol, total supply, the owner's balance and contract ownership are all correct. |
| 7 | `distributeShares()` transfers tokens to a recipient | The owner's balance falls and the recipient's rises by 100,000 ALUT; zero amounts and non-owners are rejected. |
| 8 | `ownershipPercentage()` returns the correct percentage | 100,000 ALUT returns 10, 500,000 returns 50, and the owner's remaining 400,000 returns 40. |

## 7. Conclusion

This project showed me how blockchain technology can protect a real institutional asset. Exploring Coinbase made clear how much of the crypto experience depends on regulation (KYC and AML), fees and security features such as 2FA and allowlisting, and that internal exchange purchases are not on-chain transactions. Building the contracts taught me why SHA-256 hashes make reliable fingerprints, why data on a blockchain is so hard to change, how ERC-721 and ERC-20 serve different purposes, and how small ordering decisions, like updating state before `_safeMint()`, prevent real attacks. Writing tests against the real logo file, including a tampered copy, gave me confidence that the system actually detects fakes.

With more time, I would add or improve the following:

- **Restrict registration to ALU.** Use OpenZeppelin `AccessControl`, ideally with a multi-signature wallet, so that only authorised ALU accounts can register assets and nobody can create look-alike records.
- **Link the shares to the NFT on-chain.** Store the registry address and token ID in `ALULogoToken`, or lock the logo NFT inside a vault contract, so the ALUT supply is provably backed by the registered logo.
- **Add rich metadata.** Implement `tokenURI` pointing to the logo stored on IPFS, so wallets and explorers display the image itself.
- **Improve precision and governance.** Return ownership in basis points (0.01%) so small holders do not show 0%, and use `ERC20Votes` so ALUT holders can vote on logo-licensing proposals.
- **Go public.** Deploy to the Sepolia testnet, verify the source code on Etherscan, and build the dApp in the next assignment with drag-and-drop file verification.

## 8. GitHub Repository and Video Demonstration

| Item | Link |
|---|---|
| GitHub repository (public) | https://github.com/Noorul-Ayn/alu-logo-registry-formative |
| Video demonstration | https://youtube.com/shorts/R1dF3BS3VZM |

The repository contains both contracts, the deployment and helper scripts, the eight tests, the ALU logo file, `hardhat.config.js`, the README with setup instructions and this report.

## References

African Leadership University. Official logo file, alueducation.com/wp-content/uploads/2023/05/ALU-logo-with-name-min.png (downloaded 1 November 2026).

Coinbase Help Center. "Verify your identity on Coinbase"; "Set up your 2-step verification"; "Understanding the order types". help.coinbase.com.

Coinbase. "Pricing & Fees Disclosures" and account security guidance (address book allowlisting). support.coinbase.com.

FinCEN (2013). FIN-2013-G001: Application of FinCEN's Regulations to Persons Administering, Exchanging, or Using Virtual Currencies. U.S. Department of the Treasury.

Republic of Rwanda. Law No. 023/2026 of 25/05/2026 Regulating Virtual Asset Business (summarised by ENSafrica, "Rwanda gazettes first law regulating virtual asset business", 2026).

Coinbase. Explore page and Ethereum price page (coinbase.com/explore, coinbase.com/price/ethereum), viewed 1 November 2026.

Entriken, W., Shirley, D., Evans, J. and Sachs, N. (2018). EIP-721: Non-Fungible Token Standard. eips.ethereum.org/EIPS/eip-721.

Vogelsteller, F. and Buterin, V. (2015). EIP-20: Token Standard. eips.ethereum.org/EIPS/eip-20.

NIST (2015). FIPS 180-4: Secure Hash Standard (SHS).

OpenZeppelin. Contracts 5.x documentation: ERC721, ERC20, Ownable. docs.openzeppelin.com/contracts/5.x.

Nomic Foundation. Hardhat 2 documentation. hardhat.org.